

Prueba de Caja Blanca

“Sistema de Gestión de Caja para Tags y Productos”

Integrantes:
Leopoldo Bolaños
Matias Medina
Martin Palacios

Fecha: 2026/06/19

REQ004 (Servicios TAG):

Módulo que registra de forma automatizada trámites de peajes seleccionados desde un menú desplegable, asignando de manera exacta su tarifa correspondiente según el catálogo establecido.

1. CÓDIGO FUENTE para Servicios TAG Controlador (Código)

```
package espe.edu.ec.programacaja.controller;

import espe.edu.ec.programacaja.model.CashFlowModel;
import espe.edu.ec.programacaja.view.TagServicePanel;

import javax.swing.*;
import java.util.LinkedHashMap;
import java.util.Map;

public class TagServiceController {

    private TagServicePanel view;
    private CashFlowModel cashFlowModel;

    private Map<String, Double> tagCatalog;
    private Map<String, Integer> temporalServices;

    public TagServiceController(TagServicePanel view, CashFlowModel cashFlowModel) {
        this.view = view;
        this.cashFlowModel = cashFlowModel;

        tagCatalog = new LinkedHashMap<>();
        temporalServices = new LinkedHashMap<>();

        initCatalog();
        initView();
        initEvents();
    }

    private void initCatalog() {
        tagCatalog.put("TAG", 10.00);
        tagCatalog.put("Panavial", 5.00);
        tagCatalog.put("Guayasamín", 5.00);
        tagCatalog.put("Activación", 3.00);
        tagCatalog.put("Desactivación", 3.00);
    }

    private void initView() {
        String[] servicios = tagCatalog.keySet().toArray(new String[0]);
        view.cargarServicios(servicios);
        actualizarPrecio();
        actualizarResumenTemporal();
    }

    private void initEvents() {
        view.getCbServicioTag().addActionListener(e -> actualizarPrecio());

        view.getBtnAgregar().addActionListener(e -> agregarServicioTemporal());
    }
}
```

```

view.getBtnGuardarServicios().addActionListener(e -> guardarServiciosEnCaja());
}

private void actualizarPrecio() {
    String servicio = view.getServicioSeleccionado();

    if (servicio != null && tagCatalog.containsKey(servicio)) {
        double precio = tagCatalog.get(servicio);
        view.setPrecioTexto(String.format("%.2f", precio));
    } else {
        view.setPrecioTexto("$0.00");
    }
}

private void agregarServicioTemporal() {
    String servicio = view.getServicioSeleccionado();
    int cantidad = view.getCantidad();

    if (servicio == null || servicio.isEmpty()) {
        showError("Debe seleccionar un servicio TAG.");
        return;
    }

    if (cantidad <= 0) {
        showError("La cantidad debe ser mayor a cero.");
        return;
    }

    int cantidadActual = temporalServices.getOrDefault(servicio, 0);
    temporalServices.put(servicio, cantidadActual + cantidad);

    view.reiniciarCantidad();
    actualizarResumenTemporal();
}

private void guardarServiciosEnCaja() {
    if (temporalServices.isEmpty()) {
        showError("No hay servicios TAG para guardar.");
        return;
    }

    try {
        for (Map.Entry<String, Integer> entry : temporalServices.entrySet()) {
            String servicio = entry.getKey();
            int cantidad = entry.getValue();
            double precio = tagCatalog.get(servicio);

            cashFlowModel.addTagServiceSale(servicio, cantidad, precio);
        }

        temporalServices.clear();
        actualizarResumenTemporal();

        JOptionPane.showMessageDialog(
            view,

```

```

        "Servicios TAG guardados correctamente en el cierre de caja.",
        "Servicios guardados",
        JOptionPane.INFORMATION_MESSAGE
    );

    } catch (IllegalArgumentException ex) {
        showError(ex.getMessage());
    }
}

private void actualizarResumenTemporal() {
    if (temporalServices.isEmpty()) {
        view.setResumenTemporal("No hay servicios TAG agregados.");
        return;
    }

    StringBuilder sb = new StringBuilder();
    double subtotal = 0.0;

    sb.append("SERVICIOS TAG TEMPORALES\n\n");

    for (Map.Entry<String, Integer> entry : temporalServices.entrySet()) {
        String servicio = entry.getKey();
        int cantidad = entry.getValue();
        double precio = tagCatalog.get(servicio);
        double total = precio * cantidad;

        subtotal += total;

        sb.append(String.format(
            "%s $%.2f x%d = $%.2f\n",
            servicio,
            precio,
            cantidad,
            total
        ));
    }

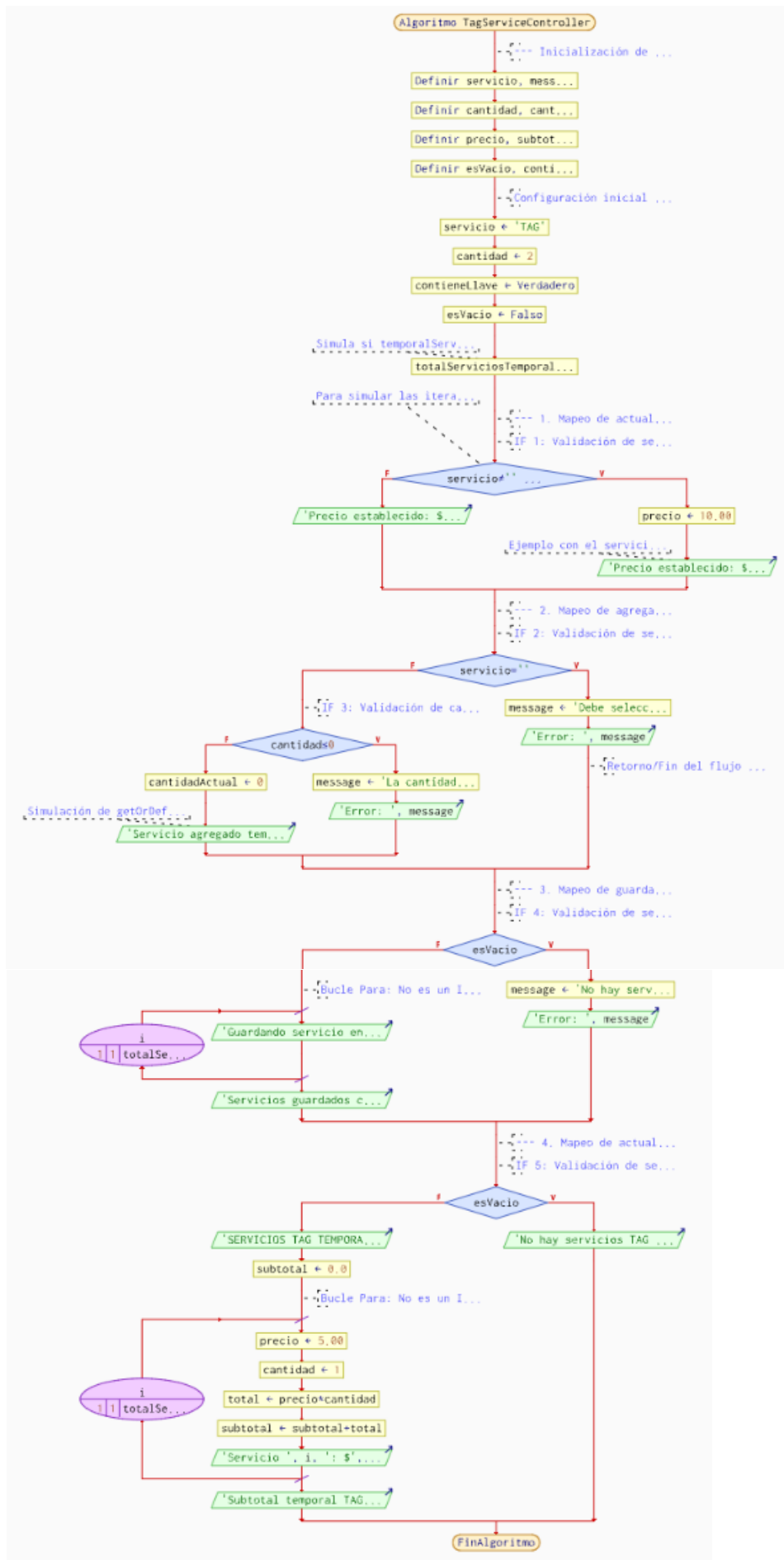
    sb.append(String.format("\nSubtotal temporal TAG: $%.2f", subtotal));

    view.setResumenTemporal(sb.toString());
}

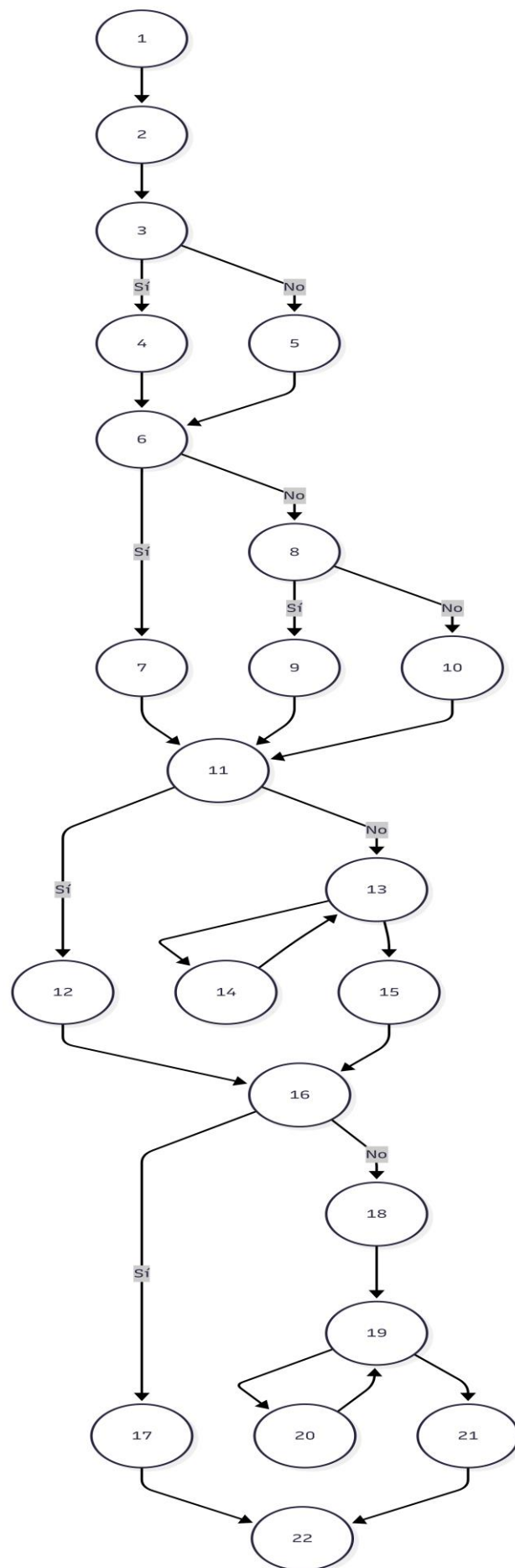
private void showError(String message) {
    JOptionPane.showMessageDialog(
        view,
        message,
        "Error",
        JOptionPane.ERROR_MESSAGE
    );
}
}

```

2. DIAGRAMA DE FLUJO (DF) Servicios TAG



3. GRAFO DE FLUJO (GF) Servicios TAG



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) Servicios TAG

Determinar en base al GF del numeral 4

RUTAS

R1: 1-2-3-4-6-7-11-12-16-17-22

R2: 1-2-3-5-6-7-11-12-16-17-22

R3: 1-2-3-4-6-8-9-11-12-16-17-22

R4: 1-2-3-4-6-8-10-11-12-16-17-22

R5: 1-2-3-4-6-7-11-13-14-13-15-16-17-22

R6: 1-2-3-4-6-7-11-12-16-18-19-20-19-21-22

5. COMPLEJIDAD CICLOMÁTICA Servicios TAG

Se puede calcular de las siguientes formas:

V/G) Nodos predcados (IF-THEN-ELSE) +1

$V(G) = 5 + 1$

$V(G) = 6$

$V(G) = A - N + 2$

$V(G) = 26 - 22 + 2$

$V(G) = 6$

DONDE:

P: Número de nodos predcado

A: Número de aristas

N: Número de nodos